



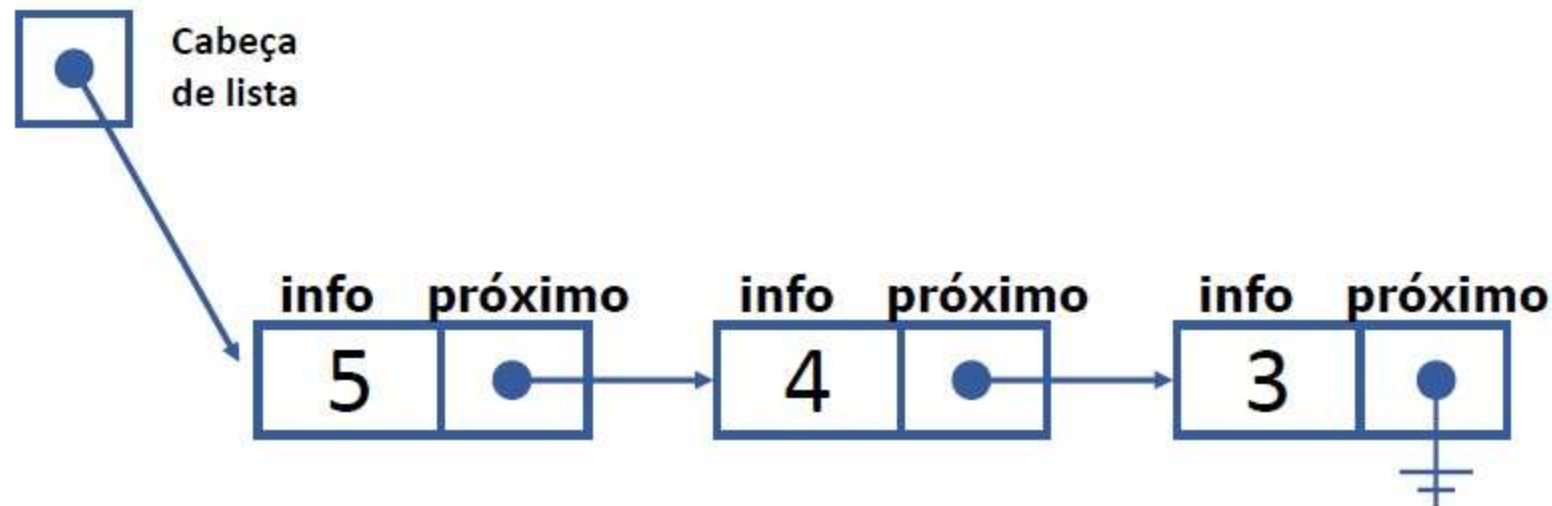
ESTRUTURA DE DADOS

Aula 7 – Lista Encadeada Simples

Prof. Evandro Luiz

LISTA ENCADEADA SIMPLES - NÓS

- Cada item de dados é incorporado em um nó;
- Cada nó possui uma referência para o próximo nó da lista;
- Um campo da própria lista contém uma referência para o primeiro nó.



LISTAS ENCADEADAS - Posição X Relacionamento

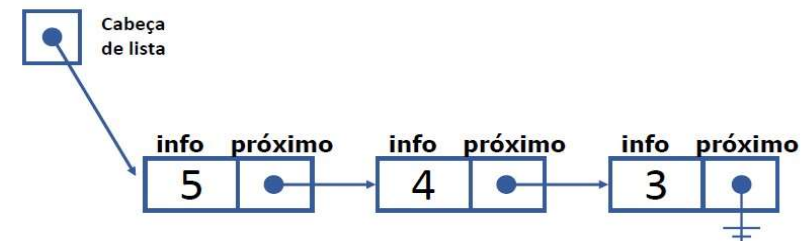
- **Vetor (posição)**

- Cada item ocupa uma certa posição;
- Cada posição pode ser acessada usando um número de índice.



- **Lista (relacionamento)**

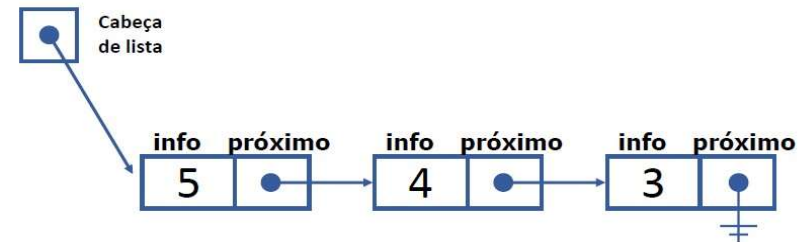
- A única maneira de encontrar um elemento é seguir a sequência de elementos;
- Um item de dados não pode ser acessado diretamente, ou seja, o relacionamento entre eles deve ser utilizado;
- Inicia com o primeiro item, vai para o segundo, então o terceiro, até encontrar o item pesquisado.



LISTAS ENCADEADAS - Posição X Relacionamento

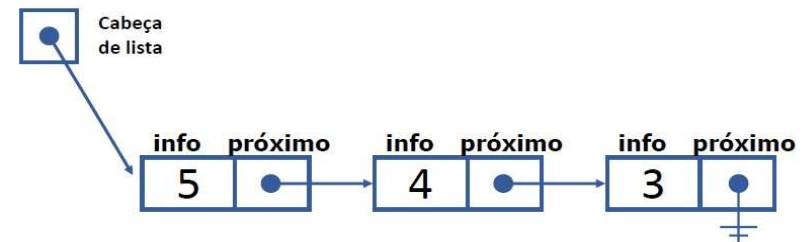


- **Inserção/Remoção Rápida:** Diferente de arrays, não é necessário deslocar elementos para inserir ou remover no início.
- **Tamanho Dinâmico:** Cresce e diminui conforme a necessidade, sem desperdício de memória pré-alocada.



LISTAS ENCADEADAS - DESVANTAGENS

- **Acesso Sequencial:** Para acessar o N-ésimo elemento, é necessário percorrer todos os anteriores.
- **Uso de Memória:** Consome memória extra para armazenar o ponteiro (próximo) de cada nó.





LISTAS ENCADEADAS APLICAÇÕES

- Implementação de Pilhas e Filas;
- Gerenciamento de Histórico (Desfazer/Refazer)
- Tabelas Hash
- Representação de Grafos (Lista de Adjacência)
- Alocação Dinâmica de Memória



LISTAS ENCADEADAS APLICAÇÕES

- **Pilhas e Filas:** Usadas quando o tamanho da estrutura é imprevisível e precisa crescer ou diminuir dinamicamente.
- **Tabelas Hash** ([Open Hashing](#)): Em tabelas hash, cada posição (bucket) pode conter uma lista encadeada para gerenciar colisões.
- **Grafos:** Representação de grafos através de listas de adjacência, ideal para grafos esparsos.



LISTAS ENCADEADAS APLICAÇÕES

Gestão de Memória e Sistemas:

- **Alocação Dinâmica de Memória:** O sistema operacional utiliza listas para monitorar blocos de memória livre (lista de livres).
- **Sistemas de Arquivos:** Manter rastreamento de blocos alocados para um arquivo.

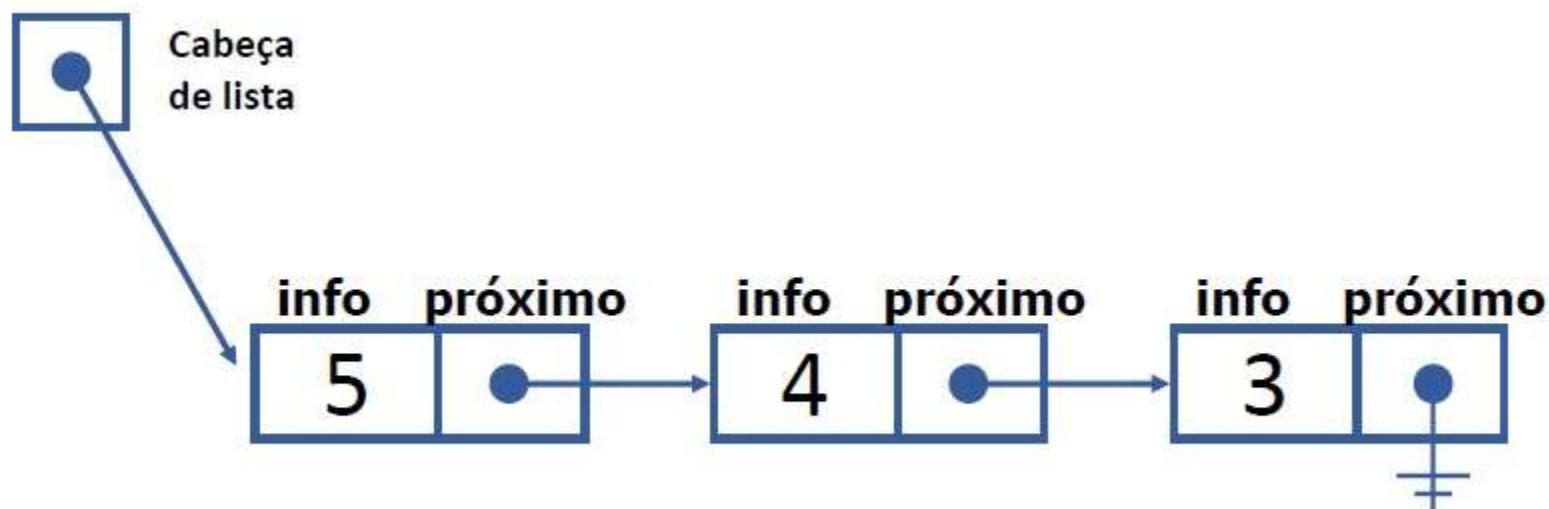
LISTAS ENCADEADAS APLICAÇÕES

Funcionalidades de Software:

- **Operações Desfazer/Refazer (Undo/Redo):** Histórico de ações em editores de texto (Word) ou imagem (Photoshop).
- **Reprodutores de Mídia:** Listas de reprodução (playlists) onde a próxima música é apontada pela atual.
- **Visualizadores de Imagens:** Navegação entre fotos próximas.

LISTAS ENCADEADAS

- Cada elemento da lista é armazenado em um objeto;
- Cada elemento da lista referencia o próximo e só é alocado dinamicamente quando é necessário;
- Para referenciar o primeiro elemento é utilizado um elemento chamado cabeça da lista.



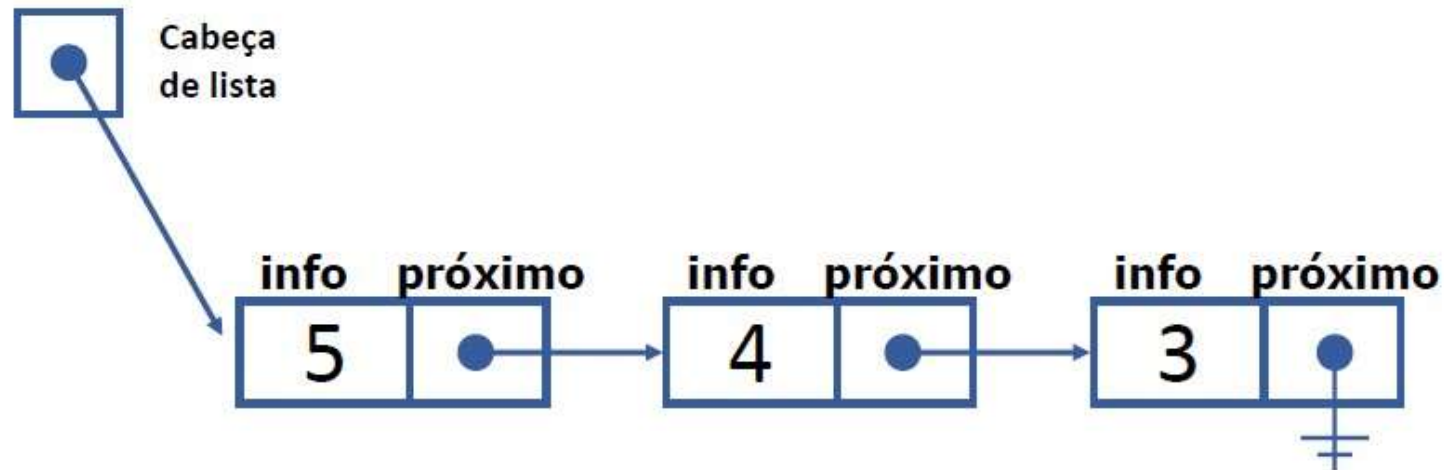
LISTAS ENCADEADAS

Lista encadeada: arranjo de memória



LISTAS ENCADEADAS - OPERAÇÕES

- Inserir no início
- Excluir do início
- Mostrar lista
- Pesquisar
- Excluir de qualquer posição





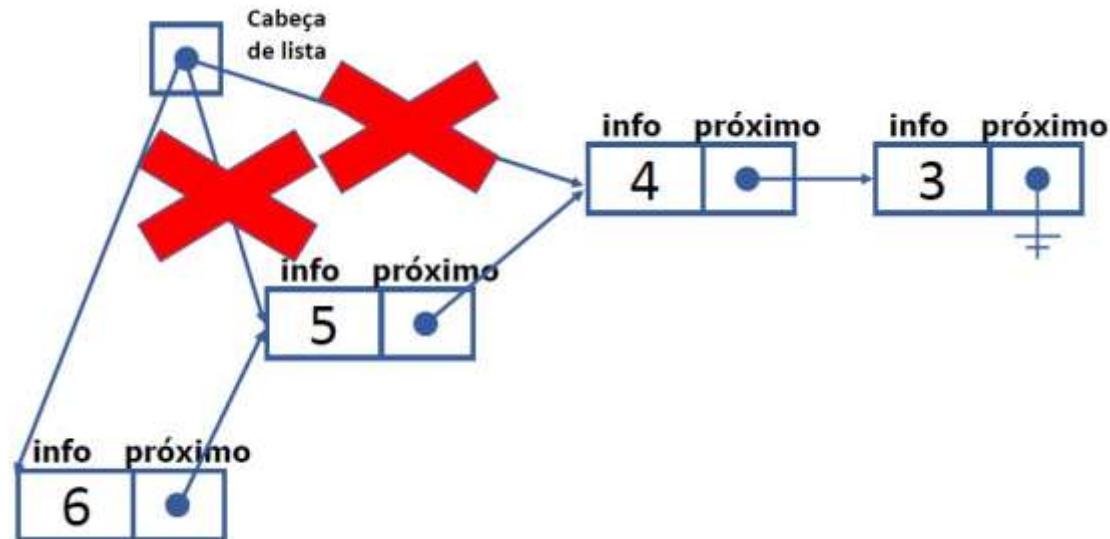
LISTAS ENCADEADAS - INSERE NO INÍCIO

- Insere um novo nó no início da lista;
- É o local mais fácil para inserir um nó;
- O próximo deste novo elemento deve apontar para quem era o primeiro da lista;
- A cabeça da lista aponta para o novo elemento.

LISTAS ENCADEADAS - INSERE NO INÍCIO

Procedimentos:

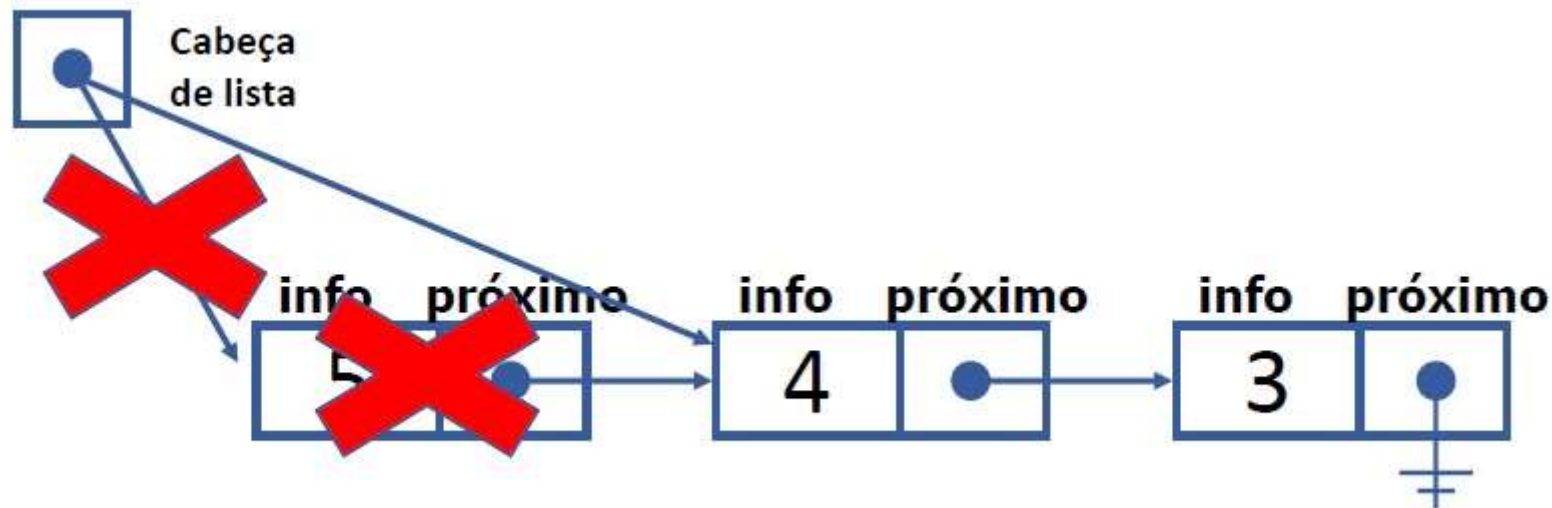
1. Criar um novo elemento (nó);
2. O próximo do novo nó criado vai apontar para o primeiro nó pré-existente da lista;
3. Destrói a conexão da cabeça da lista que aponta para o primeiro nó;
4. Faz o um novo apontamento da cabeça da lista para o nó recém criado.



LISTAS ENCADEADAS - EXCLUIR DO INÍCIO

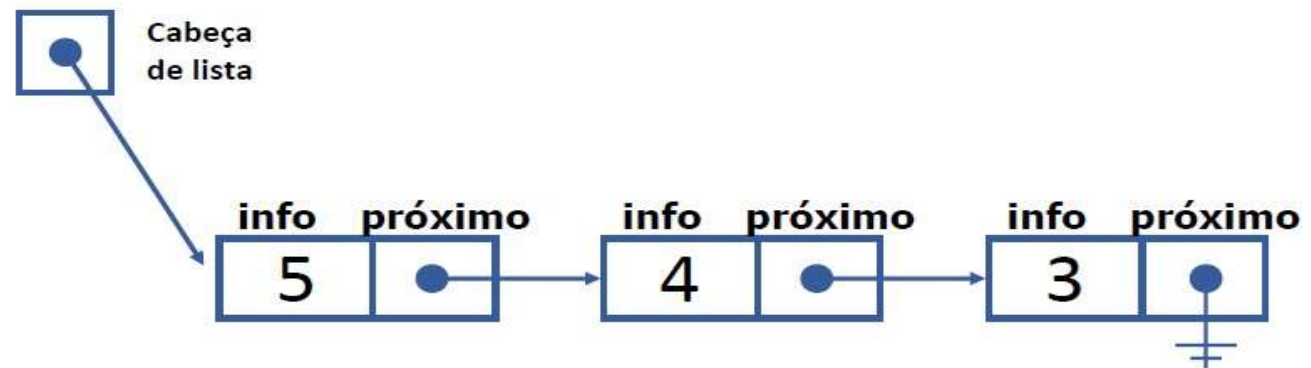
Procedimentos:

1. É o inverso do insere no início;
2. Desconecta o primeiro nó roteando o primeiro passa apontar para o segundo nó;
3. O segundo nó é encontrado por meio do campo próximo no primeiro nó.



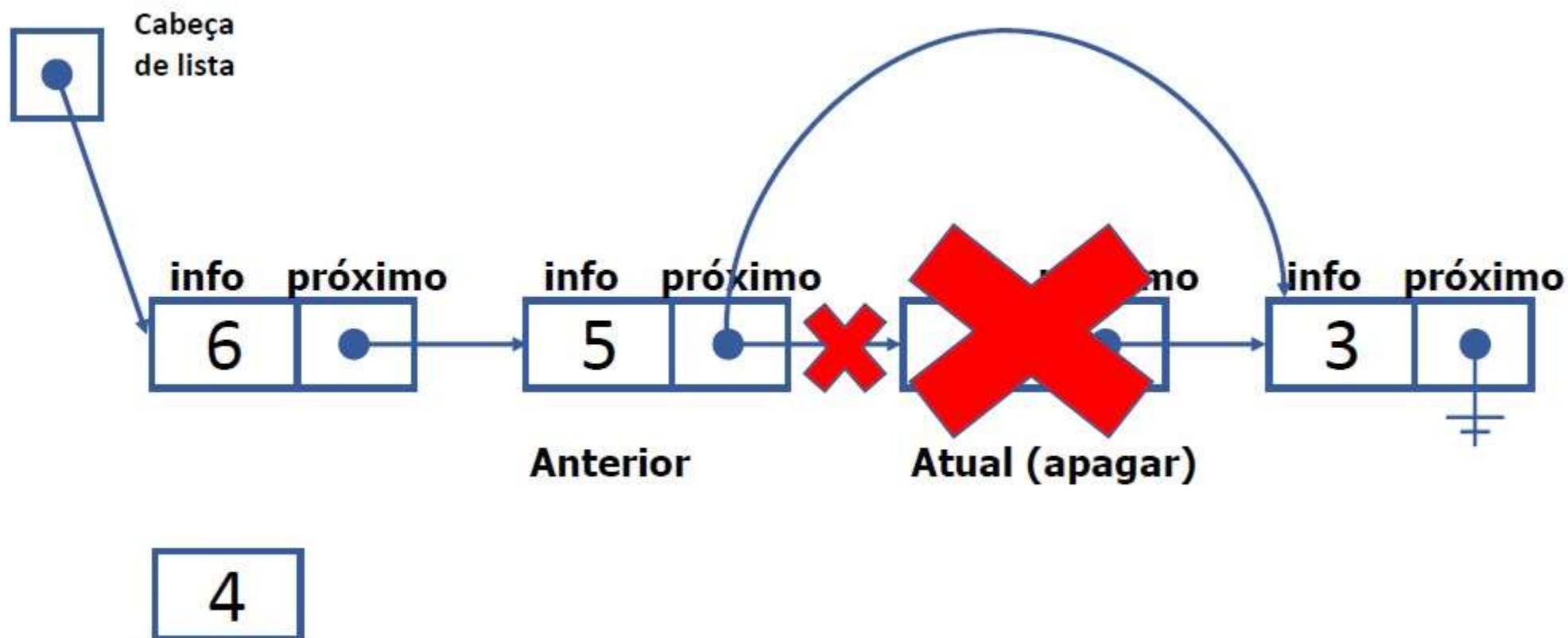
LISTAS ENCADEADAS - MOSTRAR LISTA E PESQUISAR

- Para exibir a lista, deverá ser iniciado no primeiro elemento, seguindo a sequência de referências de nó em nó;
- O final da lista é indicado pelo campo próximo no último nó apontando para null/none ao invés de outro nó;
- Os nós são percorridos e é verificado se o valor do elemento é aquele que está sendo procurado;
- Se atingir o final da lista sem encontrar o nó desejado, finaliza sem encontrar o elemento.



LISTAS ENCADEADAS - EXCLUIR DE QUALQUER POSIÇÃO

- O nó a ser eliminado deve ser localizado (pesquisa);
- Quando elimina o nó atual, terá que conectar o nó anterior ao nó seguinte.

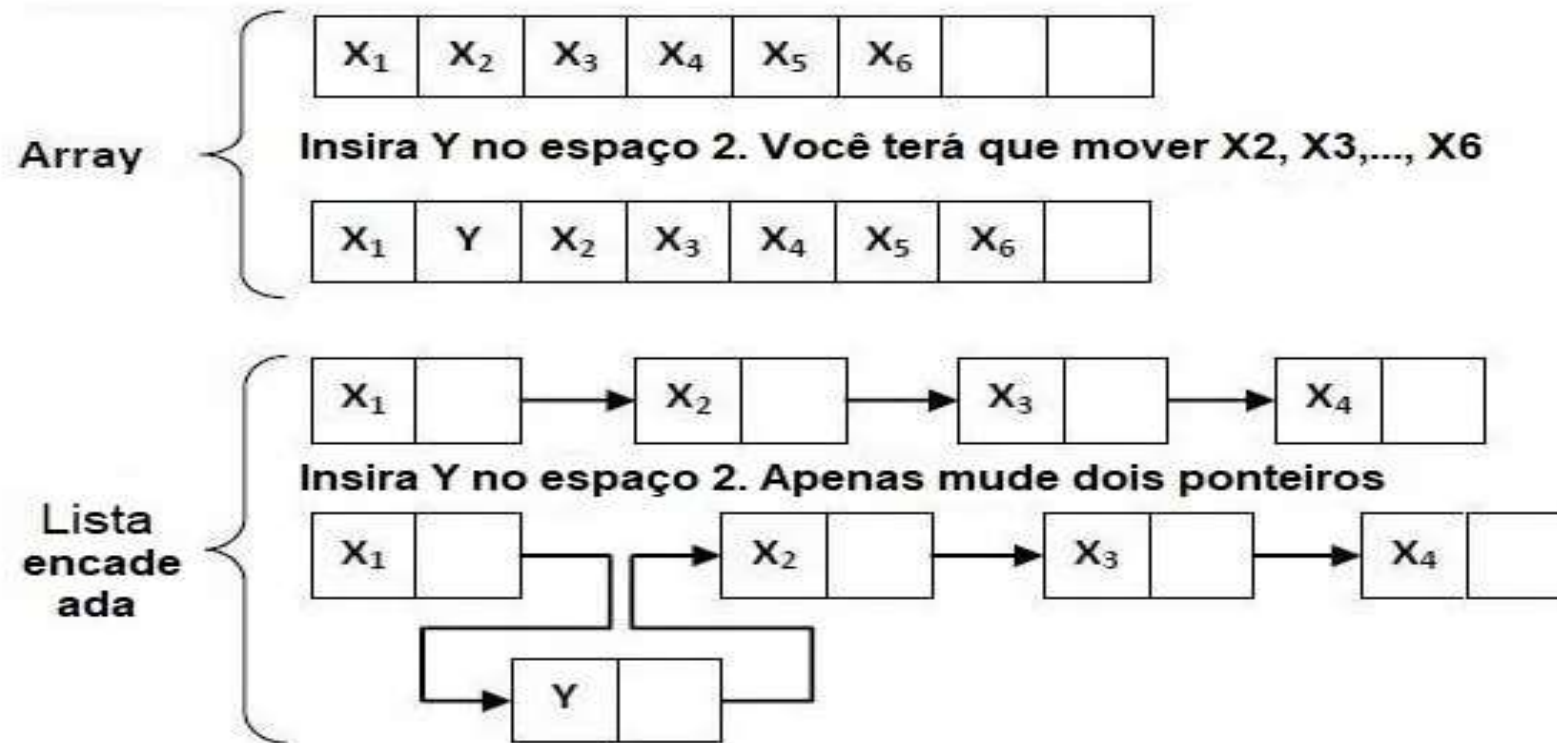


LISTAS ENCADEADAS x - ARRAY

ARRAY	Lista encadeada
O tamanho de um array é fixo	O tamanho de uma lista não é fixo
A memória é alocada estaticamente	A memória é alocada em tempo de execução (dinamicamente)
É necessário especificar o número de elementos durante a declaração (ou seja, durante a compilação)	Não é necessário especificar o número de elementos durante a declaração (ou seja, é mantida na memória dinâmica)
Ocupa menos memória que uma lista encadeada com o mesmo número de elementos	Ocupa mais memória devido a utilização de ponteiros junto à alocação do conteúdo para indicar o próximo
Inserir novos elementos na frente é potencialmente caro, pois a existência de novos elementos pode precisar mudar a estrutura	Inserir novos elementos pode ser facilmente arranjado
É possível deletar o conteúdo do elemento de um array, porém o elemento persistirá	É possível deletar um elemento

LISTAS ENCADEADAS x - ARRAY

- O quadro abaixo mostra bem as diferenças entre os dois tipos de dados; Todas essas diferenças, de alguma forma são relacionadas a como os dados se dispõem na memória, como mostra a imagem abaixo.



LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Classes:

Nó
+ valor + proximo = Nulo
+ mostraNo()

Lista Encadeada
+ primeiro = Nulo
+ inserirInicio(valor) + mostrar() + pesquisar(valor) + excluirInicio() + excluirPosição(valor)

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Algoritmo 1: Mostra Nó

mostraNo():

 imprime(valor)

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Algoritmo 2: Inserir no início

inserirInicio(valor):

novo <- Nó(valor)

novo.próximo <- primeiro

primeiro <- novo

1 – **Criação do Novo Nó** - Aqui, um novo nó é criado com um valor específico. Presumivelmente, Nó é uma estrutura que contém pelo menos o valor e um ponteiro para o próximo nó.

2 – **Linkando o novo Nó** - O ponteiro próximo do novo nó é configurado para apontar para o nó que atualmente está na cabeça da lista, que é referenciado como primeiro. Isso significa que o novo nó agora precederá todos os outros nós existentes.

3 – **Atualizando a cabeça da lista** - Aqui, o ponteiro que indica o primeiro nó da lista é atualizado para apontar para o novo nó. Isso efetivamente insere o novo nó no início da lista.

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Algoritmo 3: Mostrar Lista

mostrar():

se primeiro = Nulo **então:**

 imprime(Erro de lista vazia)

return Nulo

atual <- primeiro

enquanto atual ≠ Nulo **então:**

 atual.mostraNo()

 atual <- atual.próximo

1 – **Verificação da Lista Vazia** – Verifica se o ponteiro primeiro é Nulo, isso indica que a lista esta vazia.

2 – **Interação pela Lista** – A variável atual é definida como sendo o primeiro elemento da lista.

Um Loop enquanto atual for diferente de nulo é usado para percorrer a lista.

Dentro do Loop cada Nó (representado por Atual) chama o método mostraNo para exibir sua informação.

A seguir a variável atual é atualizada para o próximo nó.

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Algoritmo 4: Excluir no Início da Lista

excluirInicio():

se primeiro = Nulo **então:**

 imprime(Erro de lista vazia)

return Nulo

temp <- primeiro

primeiro <- primeiro.próximo

return temp

1 – **Verificação da Lista Vazia** – Verifica se o ponteiro primeiro é Nulo, isso indica que a lista esta vazia.

2 – **Remoção do Primeiro Nó** – A variável temp é utilizada guardar uma referência ao primeiro nó da lista.

A linha primeiro <- primeiro.próximo redefine primeiro para o segundo Nó da lista, efetivamente removendo o primeiro Nó.

Por fim o Nó armazenado em Temp é retornado permitindo que a memória seja liberada.

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Algoritmo 5: Pesquisar na Lista

pesquisar(valor):

se primeiro = Nulo **então:**

imprime(Erro de lista vazia)

return Nulo

atual <- primeiro

enquanto atual.valor ≠ valor **então:**

se atual.próximo = Nulo **então:**

return Nulo

senão:

 atual <- atual.proximo

return atual

A variável atual é iniciada com o primeiro nó da lista.

Um loop é utilizado para percorrer a lista até que o valor do nó atual (atual.valor) seja igual ao valor procurado.

Dentro do loop, há uma verificação para verificar se o próximo nó (atual.próximo) é Nulo:

Se for Nulo, significa que o valor não está presente na lista, e Nulo é retornado.

Caso contrário, atual é atualizado para atual.próximo, continuando a busca.

Quando o valor é encontrado, o nó atual é retornado.

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

Algoritmo 6: Excluir em qualquer posição da Lista

excluirQualquer(valor):

se primeiro = Nulo **então:**

imprime(Erro de lista vazia)

return Nulo

atual <- primeiro

anterior <- primeiro

enquanto atual.valor ≠ valor **então:**

se atual.próximo = Nulo **então:**

return Nulo

senão:

anterior <- atual

atual <- atual.próximo

se atual = primeiro **então:**

primeiro <- primeiro.próximo

senão:

anterior.próximo <- atual.próximo

return atual

–Inicialização: Variáveis atual e anterior são inicializadas para começar a travessia pela lista.

Laço de Pesquisa: Um loop continua enquanto atual não é Nulo e o valor do nó atual não corresponde ao valor que estamos buscando (valor). Dentro do loop, as variáveis anterior e atual são atualizadas para percorrer a lista.

Verificação Final: Se chegar ao final da lista (atual se torna Nulo) e o valor não foi encontrado, a função retorna Nulo.

Remoção do Nó: Se o nó a ser removido é o primeiro da lista, o ponteiro primeiro é atualizado para o próximo nó. Se for um nó do meio ou do fim, ajusta-se o ponteiro do nó anterior para o próximo do nó atual.

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

[1] → [5] → [3] → [2] → Nulo

excluirQualquer(valor):

se primeiro = Nulo **então:**

imprime(Erro de lista vazia)

return Nulo

atual <- primeiro

anterior <- primeiro

enquanto atual.valor ≠ valor **então:**

se atual.próximo = Nulo **então:**

return Nulo

senão:

anterior <- atual

atual <- atual.próximo

– Se não existir nenhum elemento, não há o que remover.

Inicializa os ponteiros,

Atual : Percorre a lista.

Anterior : Guarda o nó anterior ao atual

Percorre a lista até encontrar o valor.

O que acontece aqui ?

- Vai caminhando nó por nó

- Se chegar ao final sem encontrar, retorna Nulo

- Caso contrário para quando encontrar o valor

LISTAS ENCADEADA SIMPLES - IMPLEMENTAÇÃO

[1] → [5] → [3] → [2] → Nulo

se atual = primeiro **então:**

primeiro <- primeiro.próximo

Antes:

[1] → [5] → [3] → [2]

Removendo 1: Depois:

[5] → [3] → [2]

senão:

anterior.próximo <- atual.próximo

REMOVER ELEM. NO MEIO OU FIM

Antes: [1] → [5] → [3] → [2]

Removendo 3:

Depois: [1] → [5] → [2]

return atual

Remove o Elemento

Agora existem dois casos importantes.